

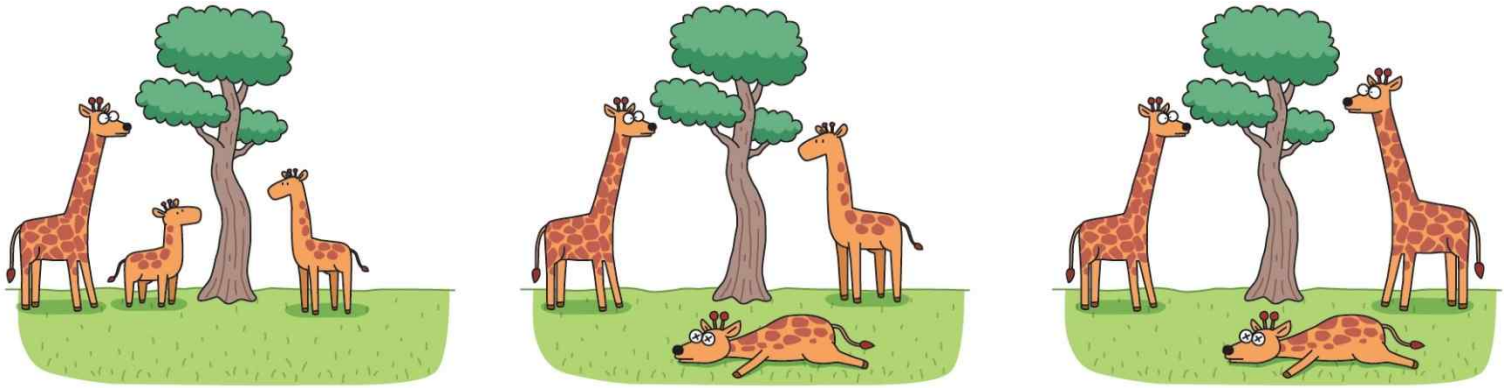
제8장 유전자 알고리즘

학습 목표

- 유전자 알고리즘의 기본 개념을 이해한다.
- 유전자 알고리즘을 여러 가지 문제에 응용해본다.
- 유전자 프로그래밍의 작동방식을 살펴본다.

01 자연계에서의 진화

- 유전자 알고리즘은 자연계의 진화에서 영감을 얻은 최적화 방법이다. 여러 후보 해를 평가하고 선택, 교차, 돌연변이를 반복하여 더 나은 해를 탐색한다.



02 유전자 알고리즘

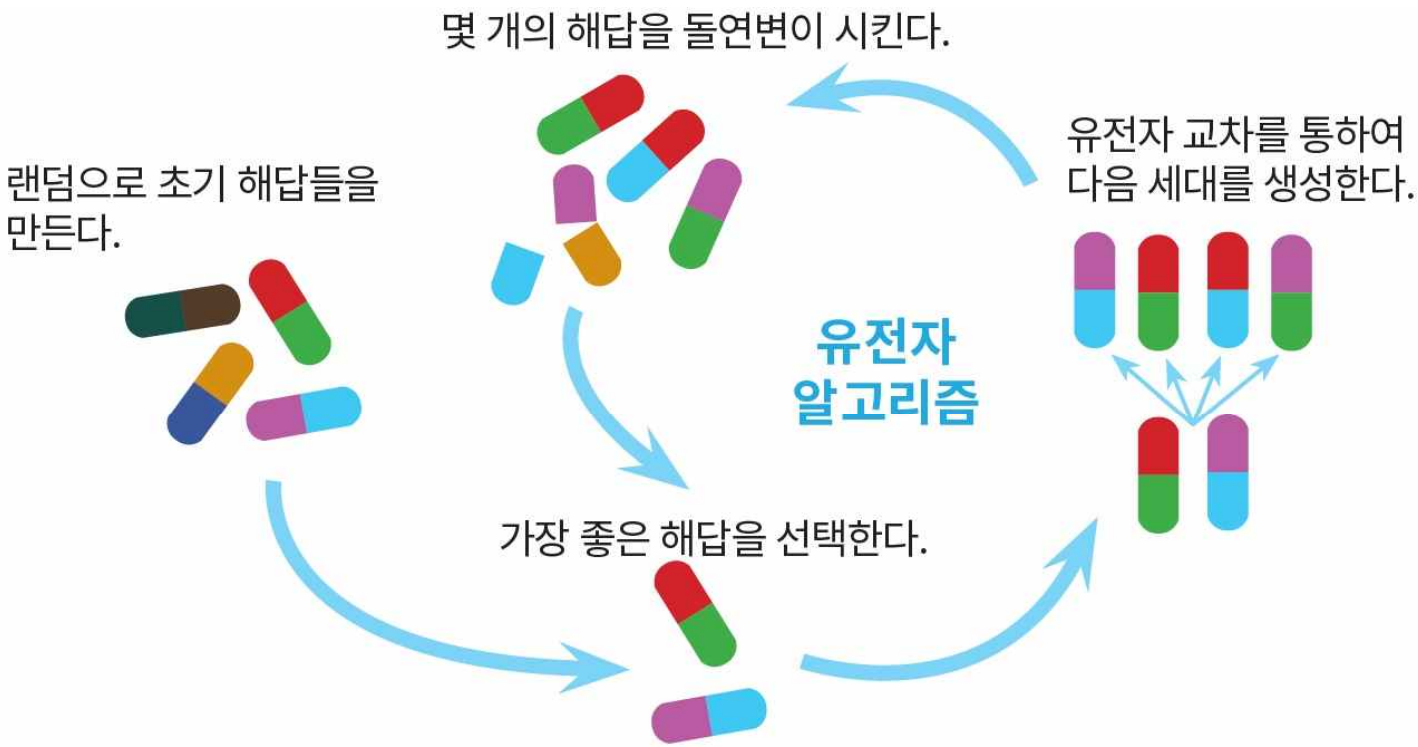


그림 8-3 유전자 알고리즘의 절차

염색체



그림 8-4 유전자의 구조

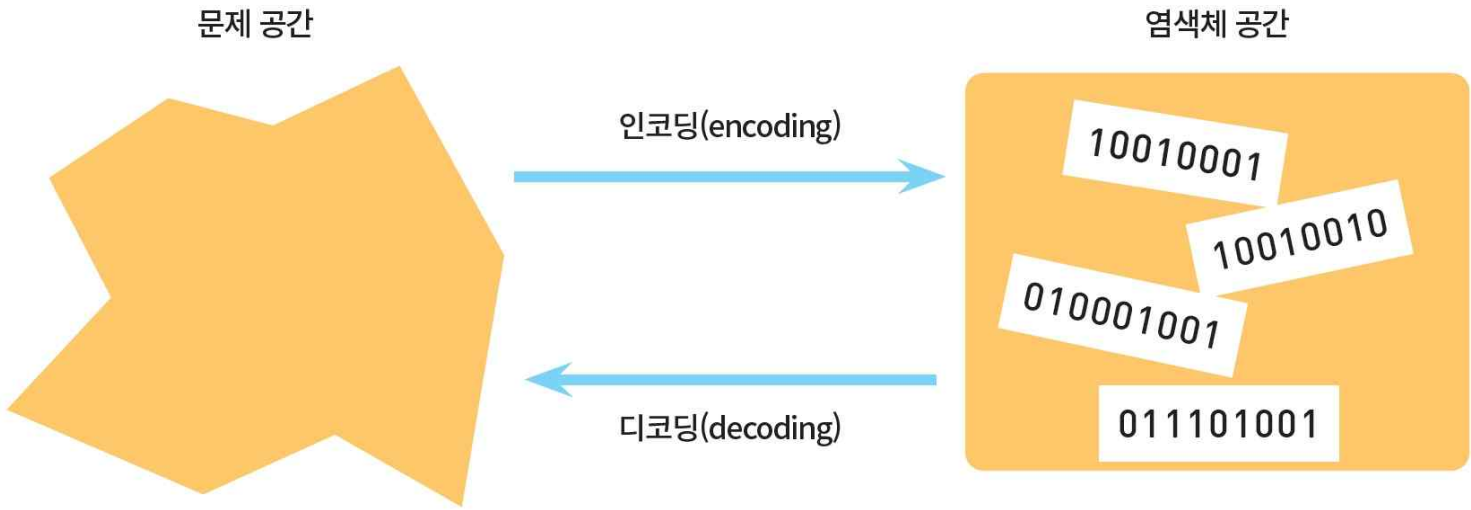


그림 8-5 인코딩과 디코딩

해를 염색체로 표현하기

정수 0~31 → 5비트: 01101은 13

8-퀸 → 각 행의 퀸이 놓인 열을 8개 정수로

TSP → 모든 도시를 한 번씩 나열한 순열

프로그램 → 연산자와 입력값으로 만든 트리

표현에 맞는 교차·돌연변이를 골라야 한다

평가함수, 적합도, 교차

- 평가 함수(evaluation function)는 현재의 염색체가 얼마나 문제를 잘 해결하고 있는지를 나타내는 적합도(fitness)를 반환
- 재생산이 일어나면 부모 염색체들의 일부는 교환된다. 이러한 연산을 교차(crossover)
- 돌연변이 연산자는 염색체에서 랜덤한 위치의 유전자의 값을 바꾼다.

유전자 알고리즘의 의사 코드

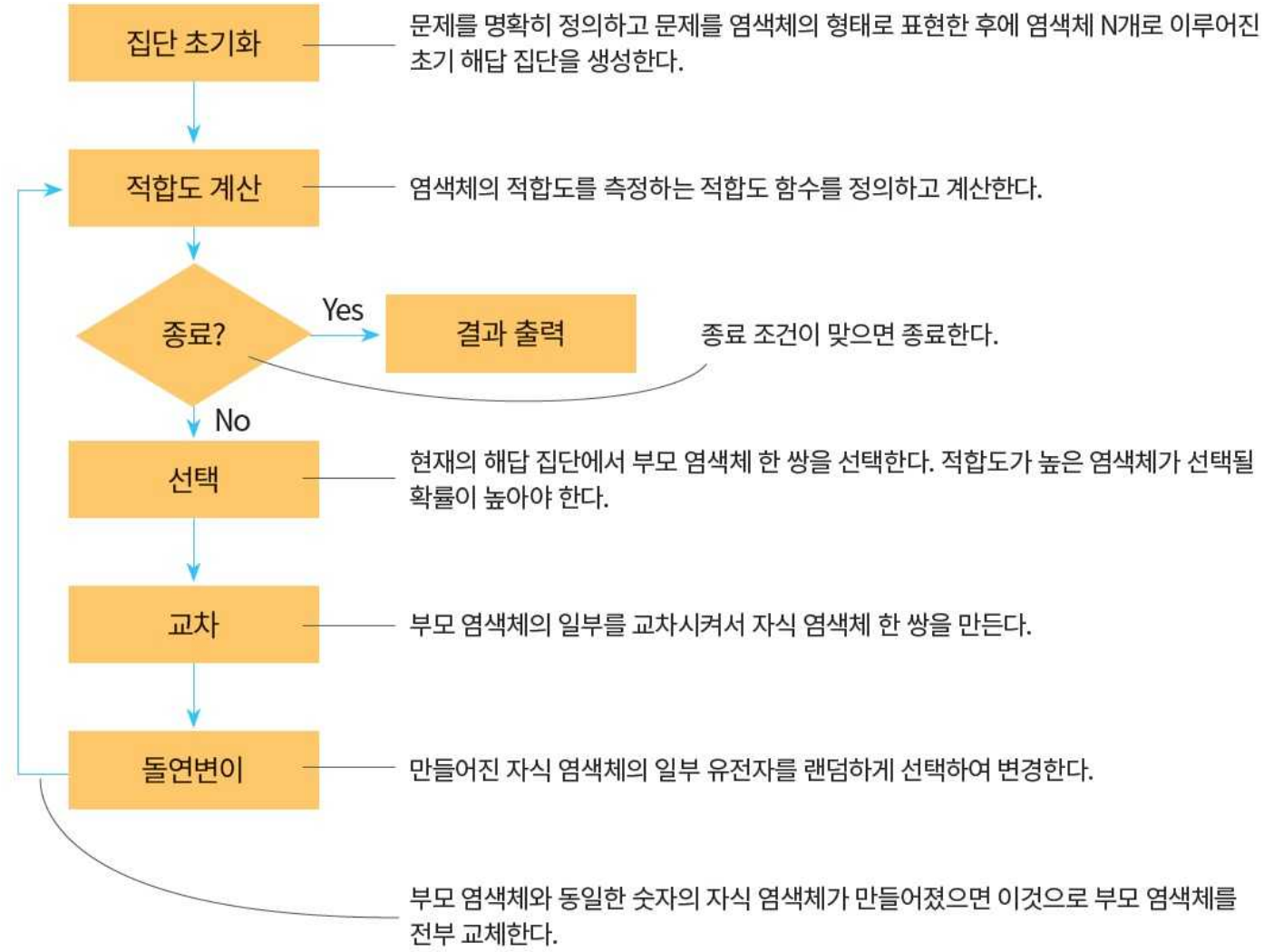
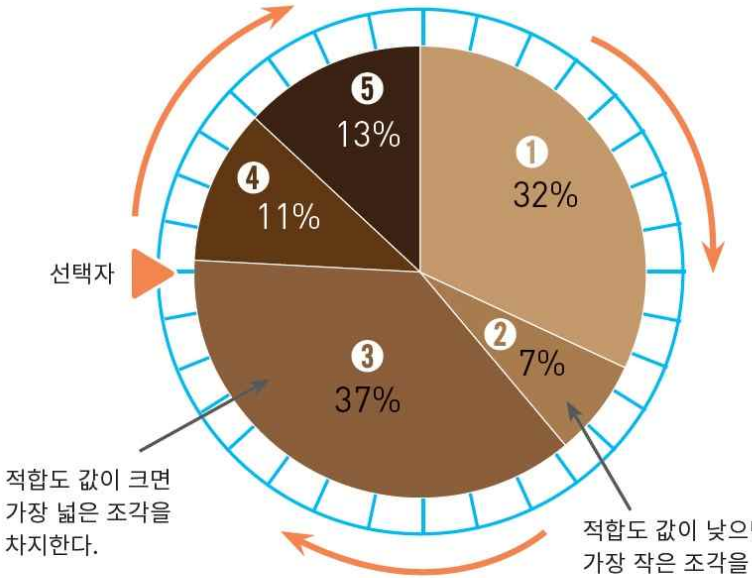


그림 8-6 유전자 알고리즘의 순서도

선택 연산자

- 재생산을 위해 부모를 선택한다. 룰렛 휠 선택(roulette-wheel selection)은 적합도가 높은 후보에 더 큰 선택 확률을 준다.



염색체	확률값
C1	32%
C2	7%
C3	37%
C4	11%
C5	13%

그림 8-7 룰렛 휠 알고리즘

룰렛 선택을 숫자로 읽기

적합도: [169, 576, 16, 361], 합계 1122

선택 확률: [0.151, 0.513, 0.014, 0.322]

누적 구간: [0,.151), [.151,.664), [.664,.678), [.678,1)

높은 적합도는 선택 기회가 크지만 확정은 아니다

교차 연산자

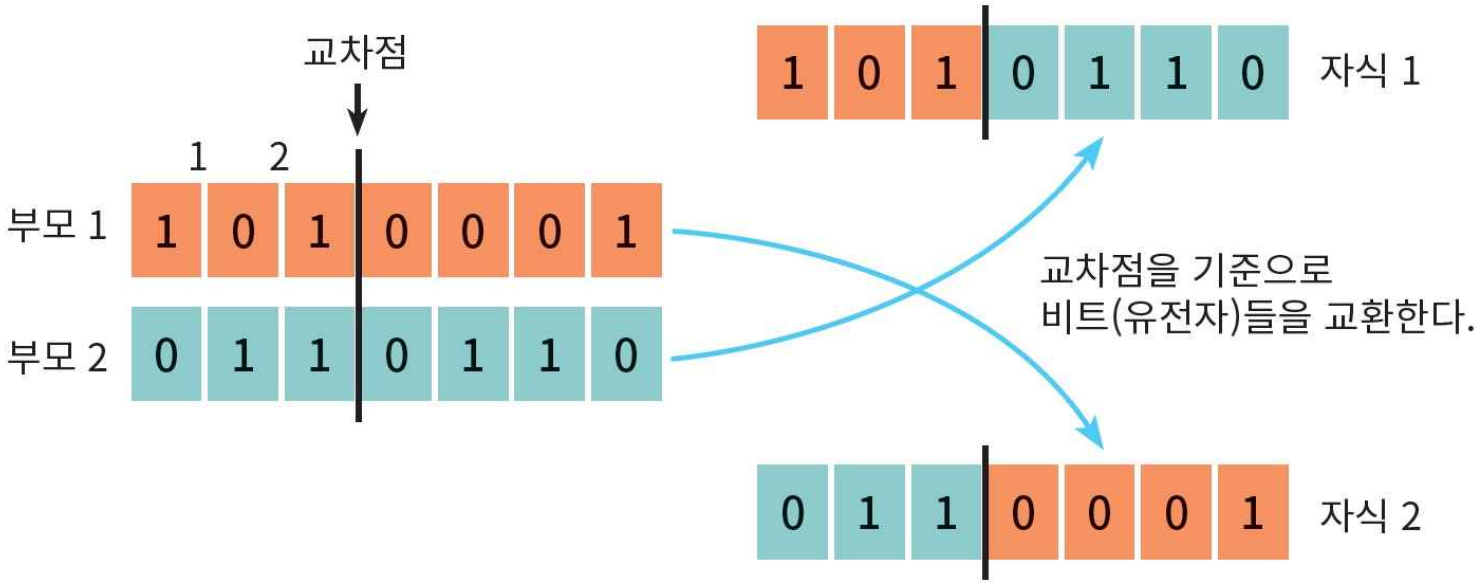


그림 8-8 교차 연산

실습 | 두 스킨 옵션을 교차하면?

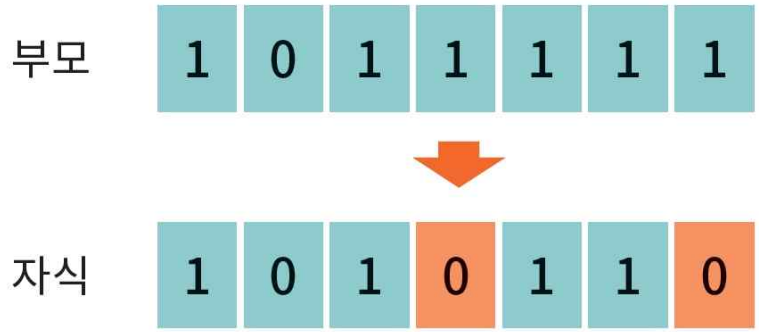
부모 A = 01101 (13), B = 11010 (26)

세 번째 비트 뒤에서 한 점 교차

두 자손의 비트열과 십진수는?

자손이 부모보다 항상 우수할까?

돌연변이 연산자



돌연변이 확률의 단위

비트별 확률 $p=0.1$, 염색체 길이 $L=5$

한 비트도 바뀌지 않을 확률: $(1-p)^5 \approx 0.590$

하나 이상 바뀔 확률: $1-(1-p)^5 \approx 0.410$

개체별 돌연변이 10%와 구별

비트 뒤집기: 0↔1

의사 코드

```
GeneticAlgorithm(population, FitnessFunc)
{
        repeat
                new_population ← []
                for i=1 to size(population) do
                        father ←select(population, FitnessFunc)
                        mother ←select(population, FitnessFunc)
                        child ← crossover(father, mother)
                        if (난수 < 변이_확률) then child←mutate(child)
                        new_population ←new_population + child
                population = new_population
        until 충분히 적합한 개체가 얻어지거나 충분한 반복횟수가 지나면
        return 가장 적합한 개체
}
```

03 유전자 알고리즘의 예제

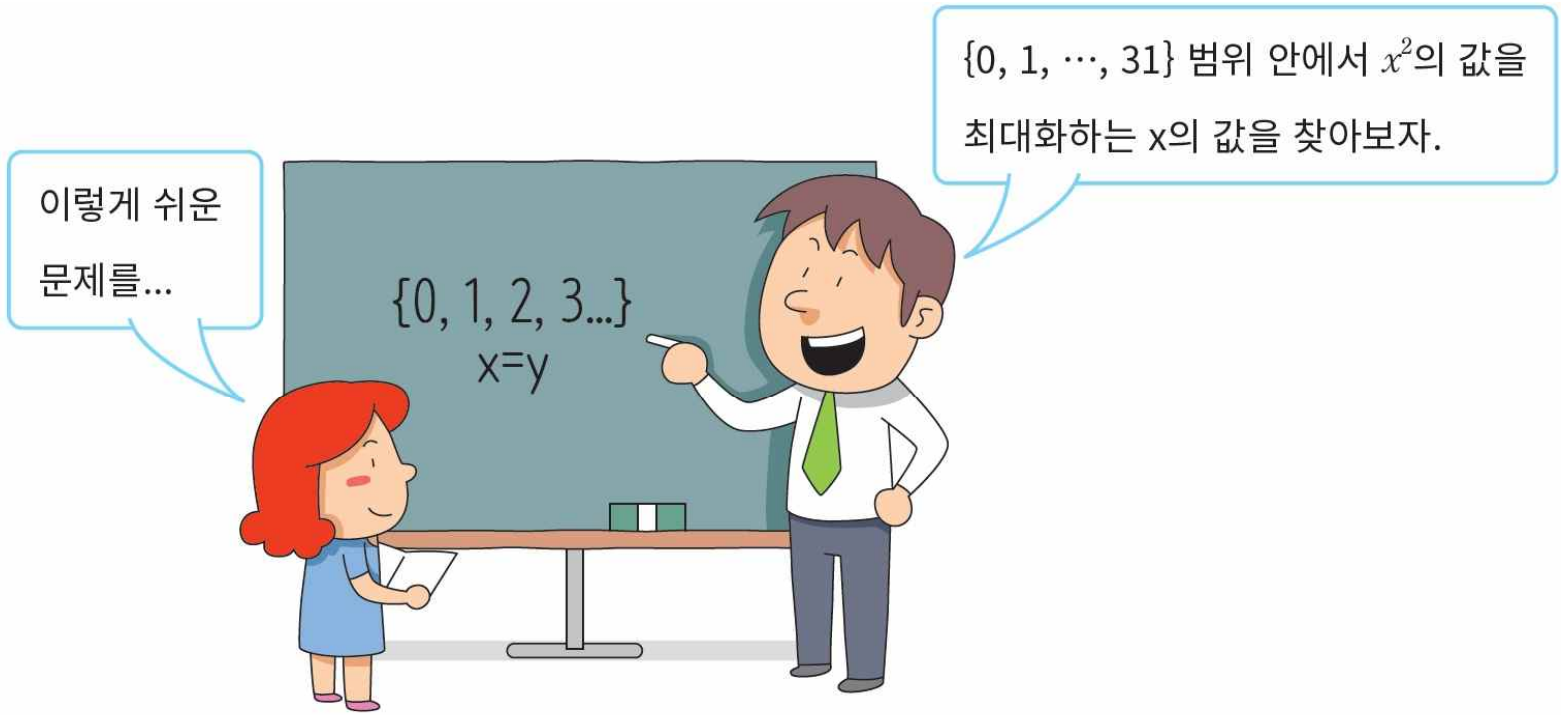


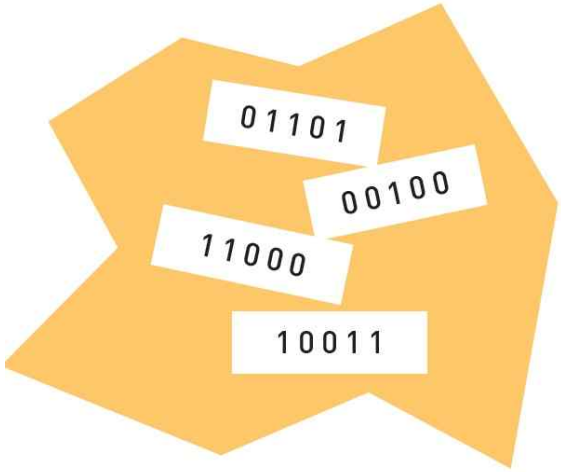
그림 8-10 유전자 알고리즘 예제

유전자 알고리즘의 예제

- 표현: 0~31을 5비트 이진수로 나타낸다.
- 모집단: 계산을 쉽게 하려고 4개체를 사용한다.
- 연산 설정: 교차 확률 0.7, 비트별 돌연변이 확률 0.1.

초기 염색체

- 랜덤하게 생성한 1과 0을 이용하여 문자열 4개를 생성



적합도 계산

염색체 번호	초기 염색체	x값	적합도 $f(x) = x^2$	적합도 비율
1	0 1 1 0 1	13	169	15%
2	1 1 0 0 0	24	576	51%
3	0 0 1 0 0	4	16	1%
4	1 0 0 1 1	19	361	32%
합계			1,122	

룰렛 휠 선택 방법

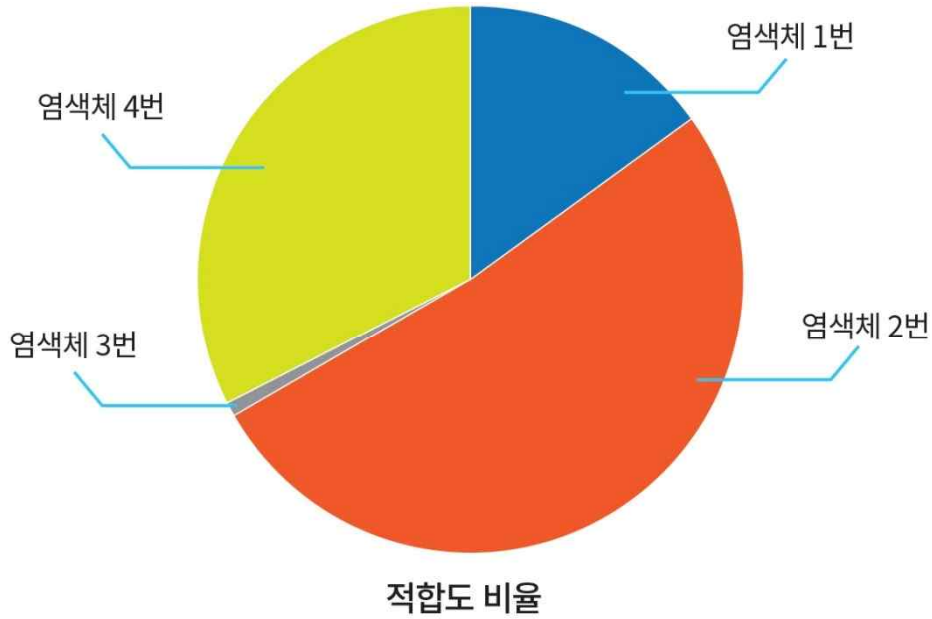


그림 8–12 룰렛 휠

예에서는 처음 두 번의 회전으로 1번과 2번이 선택되고, 다음 2번의 회전으로 2번과 4번이 선택되었다고 가정한다.

실습 | 룰렛 눈금 r=0.70

적합도 [169, 576, 16, 361]

각 후보의 누적 구간을 구한다

0 이상 1 미만의 난수 r=0.70은

몇 번째 부모를 고르는가?

교차 연산

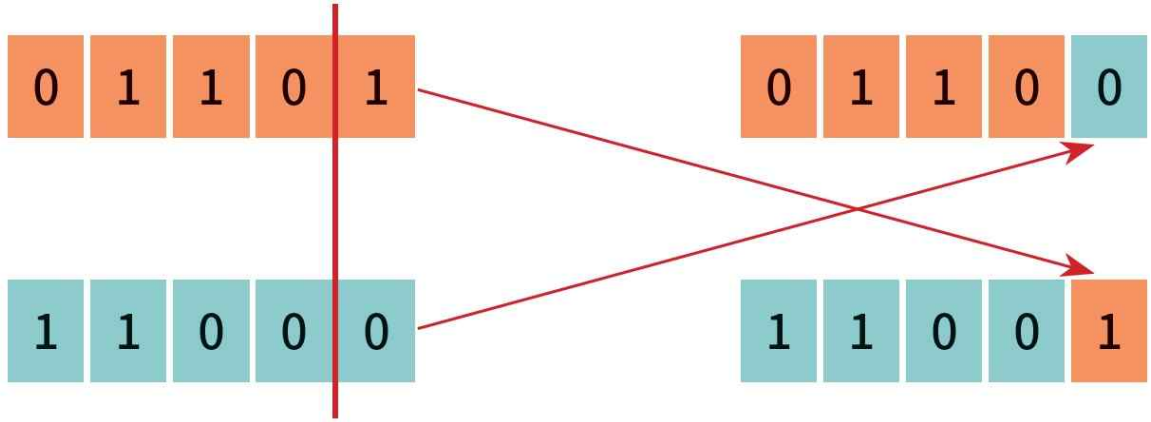


그림 8-13 교차 연산

교차 연산 후의 적합도

염색체 번호	초기 염색체	교차점	자식 염색체	x 값	적합도
1	0 1 1 0 | 1	4	0 1 1 0 0	12	144
2	1 1 0 0 | 0	4	1 1 0 0 1	25	625
2	1 1 | 0 0 0	2	1 1 0 1 1	27	729
4	1 0 | 0 1 1	2	1 0 0 0 0	16	256
합계					1,754

돌연변이 연산이 필요한 이유

- 지역 최대점 문제에 빠지는 것을 방지

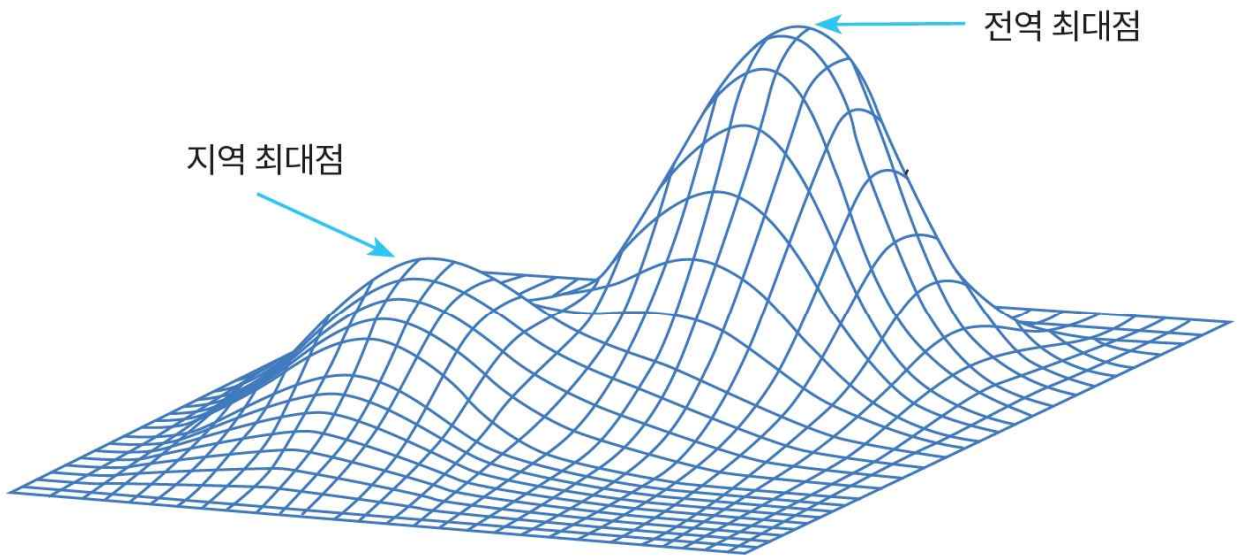


그림 8–15 지역 최대점 문제

돌연변이

새로운 염색체 번호	자식 염색체	돌연변이 위치	자식 염색체	x 값	적합도
1	0 1 1 0 0	4	0 1 1 1 0	14	196
2	1 1 0 0 1	0	1 1 0 0 1	25	625
3	1 1 0 1 1	0	1 1 0 1 1	27	729
4	1 0 0 0 0	0	1 0 0 0 0	16	256
합계					1,806

세대 교체와 종료 조건

부모 선택 → 자손 생성 → 새 세대 평가

엘리트 보존: 현 세대의 최고 개체를 복사

종료: 목표 적합도, 세대 상한, 개선 정체

평가 예산과 여러 난수 시드도 기록

확률적 탐색은 최적해 도달을 보장하지 않는다

5비트 예제 코드의 핵심 흐름

표현: 5비트 → 정수 x, 적합도 $f(x)=x^2$

선택: 양의 적합도에 비례하는 룰렛

교차 확률 0.7, 비트별 돌연변이 0.1

최선 개체 1개를 보존하고 최대 100세대

04 유전자 알고리즘 프로그램

```
import random

POPULATION_SIZE = 4        # 개체 집단의 크기
MUTATION_RATE = 0.1        # 돌연 변이 확률
SIZE = 5                   # 하나의 염색체에서 유전자 개수

# 염색체를 클래스로 정의한다.
class Chromosome:
    def __init__(self, g=[]):
        self.genes = g.copy()      # 유전자는 리스트로 구현된다.
        self.fitness = 0       # 적합도
        if self.genes.__len__()==0: # 염색체가 초기 상태이면 초기화한다.
            i = 0
            while i<SIZE:
                if random.random() >= 0.5: self.genes.append(1)
                else: self.genes.append(0)
                i += 1

    def cal_fitness(self):     # 적합도를 계산한다.
        self.fitness = 0;
        value = 0
        for i in range(SIZE):
            value += self.genes[i]*pow(2,SIZE-1-i)
        self.fitness = value
        return self.fitness

    def __str__(self):
        return self.genes.__str__()
```

유전자 알고리즘 프로그램

```
# 염색체와 적합도를 출력한다.
def print_p(pop):
    i = 0
    for x in pop:
        print("염색체 #", i, "=", x, "적합도=", x.cal_fitness())
        i += 1
    print("")

# 선택 연산
def select(pop):
    max_value  = sum([c.cal_fitness() for c in population])
    pick    = random.uniform(0, max_value)
    current = 0

    # 룰렛휠에서 어떤 조각에 속하는지를 알아내는 루프
    for c in pop:
        current += c.cal_fitness()
        if current > pick:
            return c
```

유전자 알고리즘 프로그램

```
# 교차 연산
def crossover(pop):
    father = select(pop)
    mother = select(pop)
    index = random.randint(1, SIZE - 2)
    child1 = father.genes[:index] + mother.genes[index:]
    child2 = mother.genes[:index] + father.genes[index:]
    return (child1, child2)

# 돌연변이 연산
def mutate(c):
    for i in range(SIZE):
        if random.random() < MUTATION_RATE:
            if random.random() < 0.5:
                c.genes[i] = 1
            else:
                c.genes[i] = 0
```

유전자 알고리즘 프로그램

```
# 메인 프로그램
population = []
i=0

# 초기 염색체를 생성하여 객체 집단에 추가한다.
while i < POPULATION_SIZE:
    population.append(Chromosome())
    i += 1

count=0
population.sort(key=lambda x: x.cal_fitness(), reverse=True)
print("세대 번호=", count)
print_p(population)
count=1
```

유전자 알고리즘 프로그램

```
while population[0].cal_fitness() < 31:
    new_pop = []

    # 선택과 교차 연산
    for _ in range(POPULATION_SIZE//2):
        c1, c2 = crossover(population);
        new_pop.append(Chromosome(c1));
        new_pop.append(Chromosome(c2));

    # 자식 세대가 부모 세대를 대체한다.
    # 깊은 복사를 수행한다.
    population = new_pop.copy();

    # 돌연변이 연산
    for c in population: mutate(c)

    # 출력을 위한 정렬
    population.sort(key=lambda x: x.cal_fitness(), reverse=True)
    print("세대 번호=", count)
    print_p(population)
    count += 1
    if count > 100 : break;
```

실행 결과

```
세대 번호= 0
염색체 # 0 = [1, 1, 1, 0, 0] 적합도= 28
염색체 # 1 = [0, 1, 1, 1, 0] 적합도= 14
염색체 # 2 = [0, 1, 1, 1, 0] 적합도= 14
염색체 # 3 = [0, 0, 0, 0, 0] 적합도= 0

세대 번호= 1
염색체 # 0 = [1, 1, 1, 1, 0] 적합도= 30
염색체 # 1 = [1, 1, 1, 0, 0] 적합도= 28
염색체 # 2 = [1, 1, 1, 0, 0] 적합도= 28
염색체 # 3 = [0, 1, 1, 0, 0] 적합도= 12
...
...
세대 번호= 9
염색체 # 0 = [1, 1, 1, 1, 0] 적합도= 30
염색체 # 1 = [1, 1, 1, 0, 1] 적합도= 29
염색체 # 2 = [1, 1, 0, 1, 1] 적합도= 27
염색체 # 3 = [1, 1, 0, 0, 0] 적합도= 24

세대 번호= 10
염색체 # 0 = [1, 1, 1, 1, 1] 적합도= 31
염색체 # 1 = [1, 1, 1, 1, 1] 적합도= 31
염색체 # 2 = [1, 1, 1, 0, 1] 적합도= 29
염색체 # 3 = [1, 1, 0, 1, 0] 적합도= 26
```

실습 | 코드에서 목표 함수를 바꾼다면

x는 0~31, $f(x)=6x-x^2$

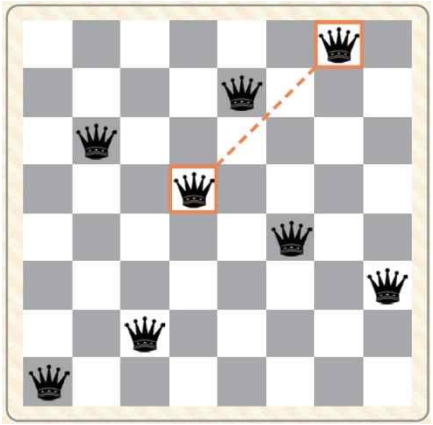
최대가 되는 x와 f(x)를 먼저 구한다

음수 적합도로 룰렛을 돌려도 되는가?

최적값을 모르면 무엇으로 종료할까?

Lab: 8-queen

- 8-queen 문제에서 적합도 함수는 어떻게 만들면 좋을까?
- 적합도 값으로 서로 공격하지 않는 queen 쌍의 개수를 사용하자. 만약 모든 queen 들이 서로를 공격하지 않는다면 8C2 = 28이 된다.
- 여기서 서로를 공격하는 queen 쌍의 개수를 뺀다. 따라서 적합도 함수는 28-h가 된다.



				8			
	7						
			6				
					5		
							4
		3					
2							
						1	

8-퀸: 두 가지 표현과 충돌 수

열 인덱스 i에 퀸 1개, 값 q_i는 행 1~8

기본 표현은 행 값이 중복되어도 후보가 된다

h = 같은 행 또는 대각선의 충돌 쌍 수

적합도 f=28−h, 해결하면 f=28

순열 표현이면 행 중복을 금지하고 연산도 변경

실습 | 4-퀸 후보 평가

열→행 표현 q=[2,4,1,3]

서로 공격하는 퀸 쌍은 몇 개인가?

전체 쌍 6개 기준 적합도 f=6−h는?

q=[1,2,3,4]는 어떨까?

Lab: TSP(Travelling Salesman Problem)

- "도시 목록과 도시 간의 거리를 제공하면 각 도시를 방문하여 출발 도시로 돌아오는 가장 짧은 경로는 무엇인가?"

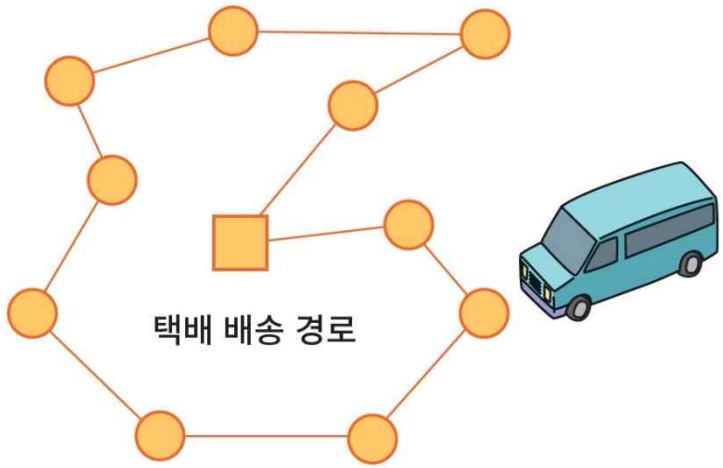


그림 8–16 택배 배송 경로

Lab: TSP

- 유전자: 하나의 도시 ((x, y) 좌표로 나타낸다.)
- 염색체: 하나의 경로
- 세대: 염색체의 모임
- 적합도 함수: 각 경로가 얼마나 좋은지를 알려주는 함수
- 돌연변이: 경로에서 도시들을 랜덤하게 바꾼다.

	0	1	2	3	4	5
0	0	5	2	6	4	5
1	5	0	7	1	3	6
2	2	7	0	6	3	3
3	6	1	6	0	2	4
4	4	3	3	2	0	5
5	5	6	3	4	5	0

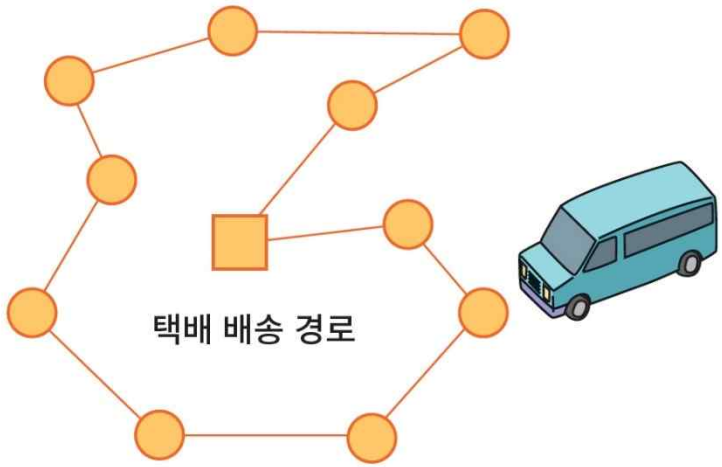


그림 8-16 택배 배송 경로

실습 | 네 도시의 경로 비교

A→B=2, B→C=3, C→D=4, D→A=5

A→C=6, B→D=7 (대칭 거리)

A-B-C-D-A와 A-B-D-C-A의 길이

어느 쪽을 선택할까?

풀이에 복귀 거리도 포함

TSP 경로의 적합도와 유효한 자손

염색체 π는 모든 도시를 한 번씩 담은 순열

경로 길이 L=연속 도시 거리의 합 + 복귀 거리

짧은 경로 선호: L 최소화 또는 1/(1+L) 최대화

일반 한 점 교차는 도시 중복·누락을 만들 수 있다

OX (ordered crossover)·PMX (partially matched crossover) 같은 순열 전용 연산 또는 복구

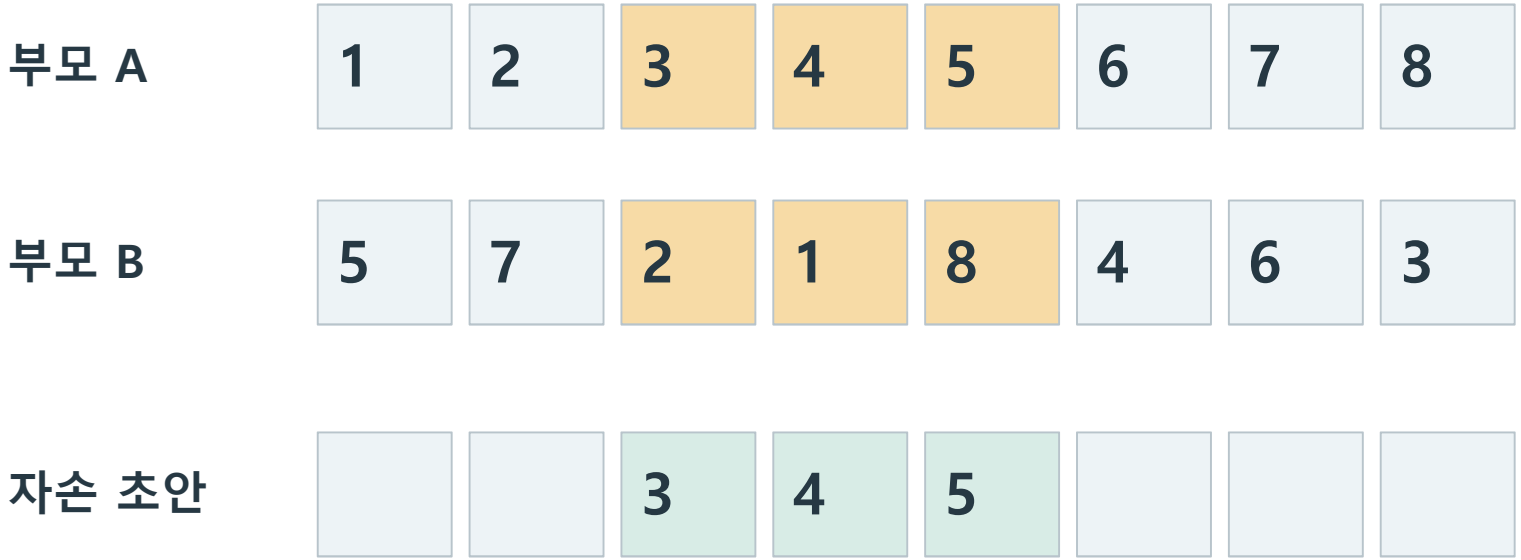
순열 교차가 필요한 이유

부모 A	1	2	3	4	5	6	7	8
부모 B	5	7	2	1	8	4	6	3
단순 교차	1	2	3	4	5	4	6	3

도시 3·4 중복, 7·8 누락: 순열 조건 위반

OX ① 연속 구간을 그대로 복사

절단 구간: 3~5번 위치 (양 끝 포함)



OX는 고정 구간의 값과 나머지 도시의 상대 순서를 결합

OX ② 부모 B의 순서로 빈칸 채우기

B의 순환 읽기: 4, 6, 3, 5, 7, 2, 1, 8

3·4·5 제외: 6, 7, 2, 1, 8

빈칸 순서	1	2	3	4	5	6	7	8
OX 자손	1	8	3	4	5	6	7	2

6→7→8→1→2번 칸에 차례로 6,7,2,1,8

PMX ① 구간과 대응 관계 만들기

부모 A	1	2	3	4	5	6	7	8
부모 B	5	7	2	1	8	4	6	3
자손 초안			3	4	5			

대응 쌍: 3↔2　　4↔1　　5↔8

자손의 바깥 칸은 B를 기준으로, 충돌 시 대응값을 추적

PMX ② 중복은 대응값으로 교체

부모 B	5	7	2	1	8	4	6	3

1번: 5→8　　6번: 4→1　　8번: 3→2

PMX 자손	8	7	3	4	5	1	6	2

1~8이 각각 한 번씩 등장한다

OX와 PMX의 자손은 어떻게 다른가

부모 A	1	2	3	4	5	6	7	8
부모 B	5	7	2	1	8	4	6	3
OX 자손	1	8	3	4	5	6	7	2
PMX 자손	8	7	3	4	5	1	6	2

OX: 순서 유지 PMX: 구간 대응으로 충돌 해결

실습 | 6개 도시에서 OX·PMX 적용

부모 A	A	B	C	D	E	F
부모 B	D	F	B	E	A	C

3~4번을 고정한 자손 A를 **OX와 PMX로** 각각 만들어 보자

GA를 사용할 때의 강점과 한계

불연속·복합 제약에서도 후보를 탐색 가능

표현과 적합도 설계가 성능을 크게 좌우

확률적 결과: 시드별 편차와 평가 횟수 확인

가능하면 정확·휴리스틱 기준선과 비교

충분한 예산을 주어도 최적해 보장은 별개

실습 | 보스전 맵의 적합도 설계

후보는 장애물·회복 아이템 배치

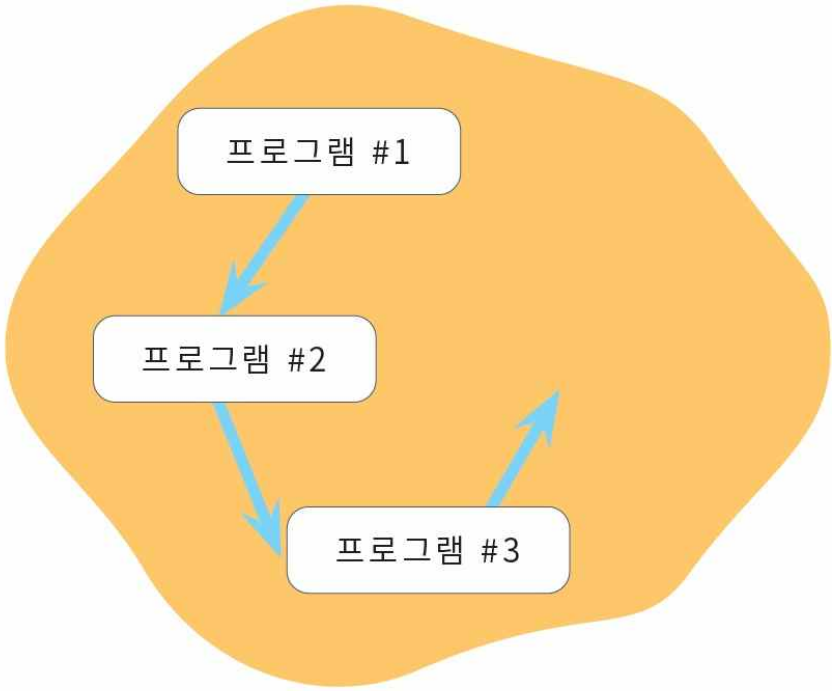
목표: 도달 가능, 전투 시간 2~4분

은폐 공간과 시야 다양성도 필요

평가 함수와 실패 조건을 어떻게 만들까?

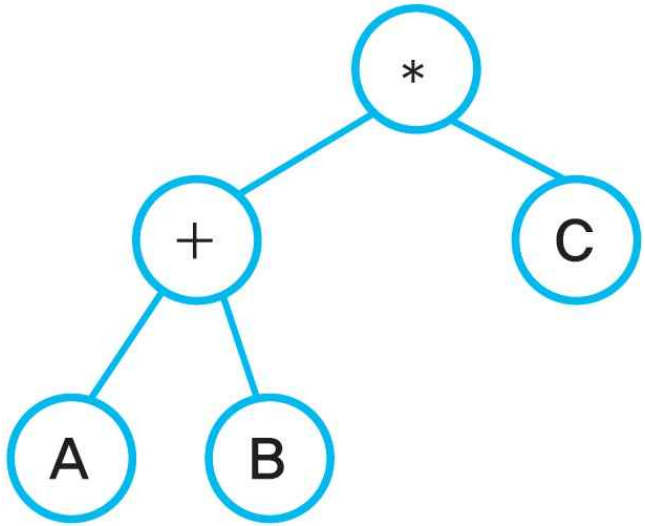
06 유전자 프로그래밍

- 유전자 프로그래밍(GP: Genetic Programming)은 유전자 알고리즘의 원리를 프로그래밍에 적용한 것이다.



유전자 프로그래밍

- GP는 함수와 입력값을 트리로 표현하고 일부 가지를 교차하거나 변이한다. LISP형 표기는 한 방법이며 Python이나 C에서도 트리를 구현할 수 있다.



GP 프로그램 트리의 실행과 안전성

내부 노드: +, −, ×, 조건문 등 함수

잎 노드: 상수·게임 센서 입력

입력에 대해 트리를 실행해 적합도를 평가

0으로 나누기와 무한 반복을 막는 연산자

트리 크기 상한으로 비대화도 제어

기본 연산

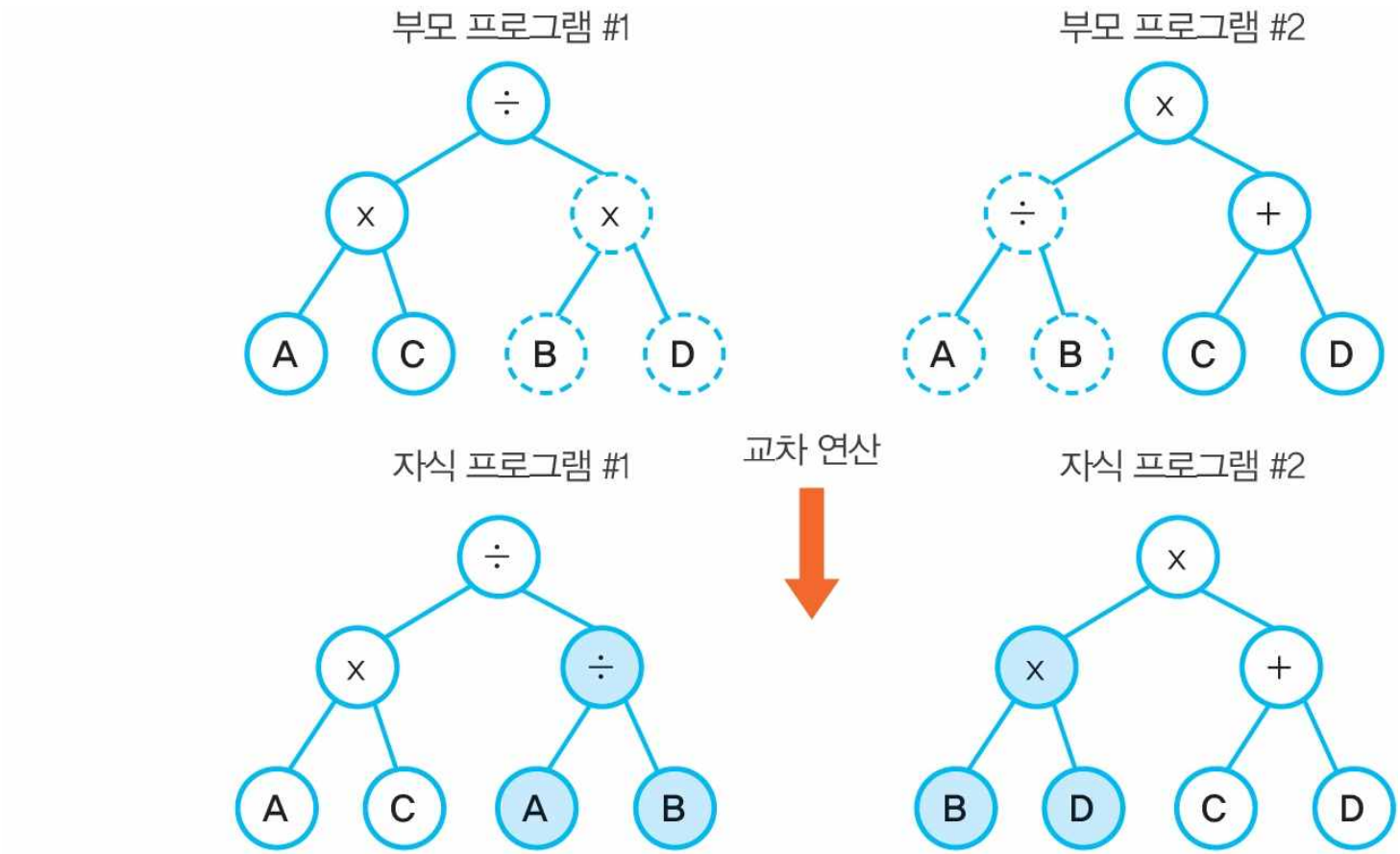


그림 8-18 수식 트리의 교차 연산

기본 연산

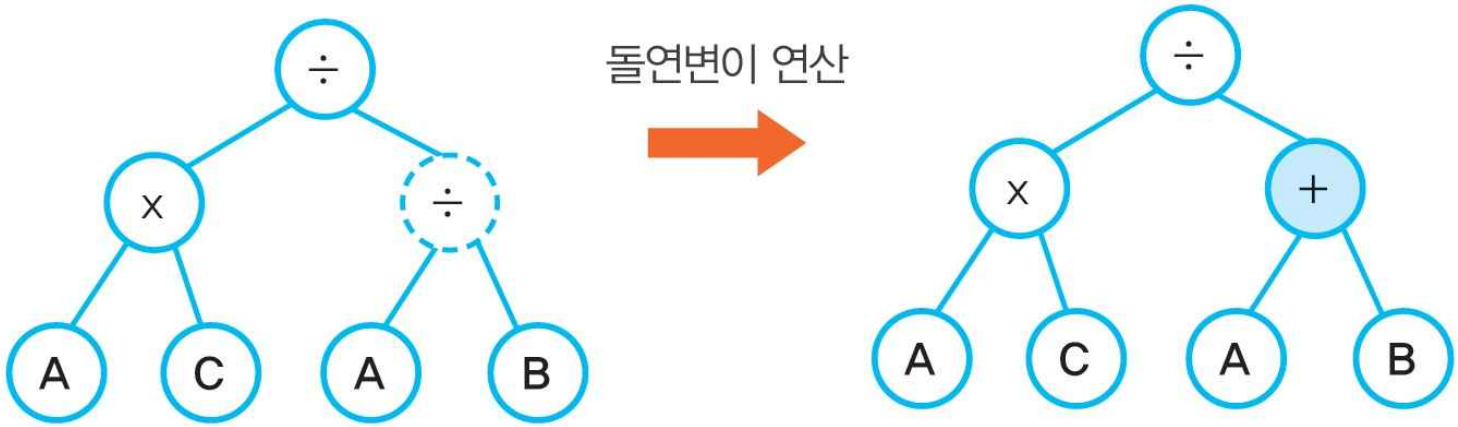


그림 8–19 수식 트리의 돌연변이 연산

실습 | 프로그램 조각 교차

부모 A: +(×(x,2),1)

부모 B: −(y,×(3,4))

A의 ×(x,2)와 B의 ×(3,4)를 교환

두 자손의 식을 써 보자

같은 반환 타입의 부분트리를 교환

GP 알고리즘

1. 초기 세대를 랜덤하게 생성한다.
2. 다음을 반복한다.
   2.1. 현재 세대의 각 프로그램을 주어진 데이터 집합에 대하여 평가한 후에 적합도를 계산한다.
   2.2. 현재 세대 안의 프로그램을 랜덤하게 **선택**하여 적합도를 비교한다.
   2.3. 다음과 같은 3가지의 유전자 알고리즘의 연산자를 적용한다.
      * 재생산 연산
      * 교차 연산
      * 돌연변이 연산
   2.4. 결과 세대 안의 모든 프로그램을 평가한다.
3. 기준이 만족될 때까지 반복한다.

벽을 따라가는 청소로봇을 위한 GP

- 센서입력 :

  n, ne, e, se, s, sw, w, nw

- 기본 함수 :

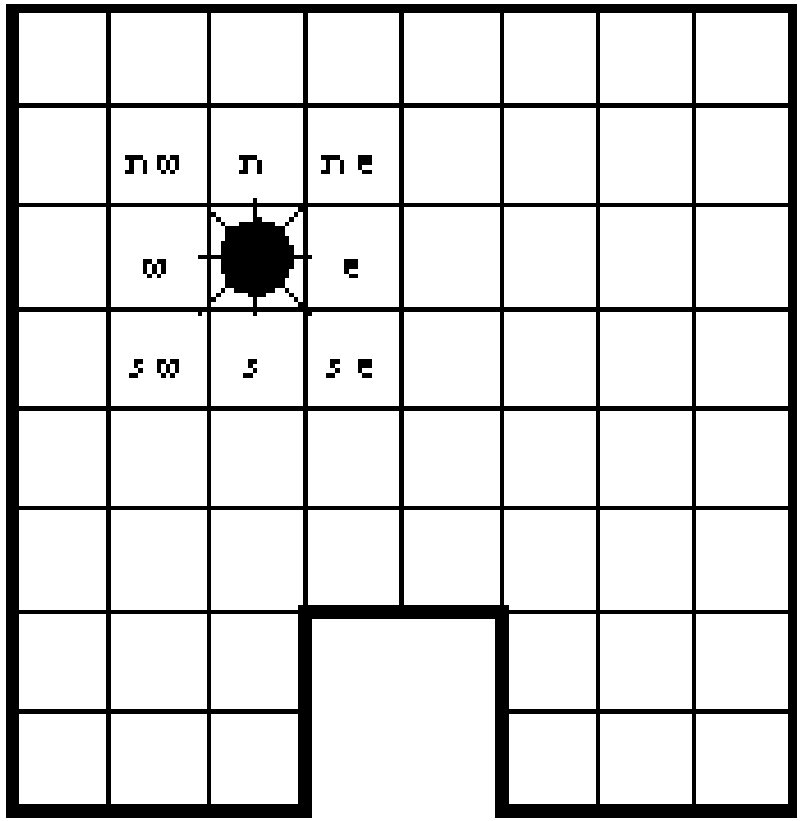
  4가지 기본 함수

  (AND, OR, NOT, IF)

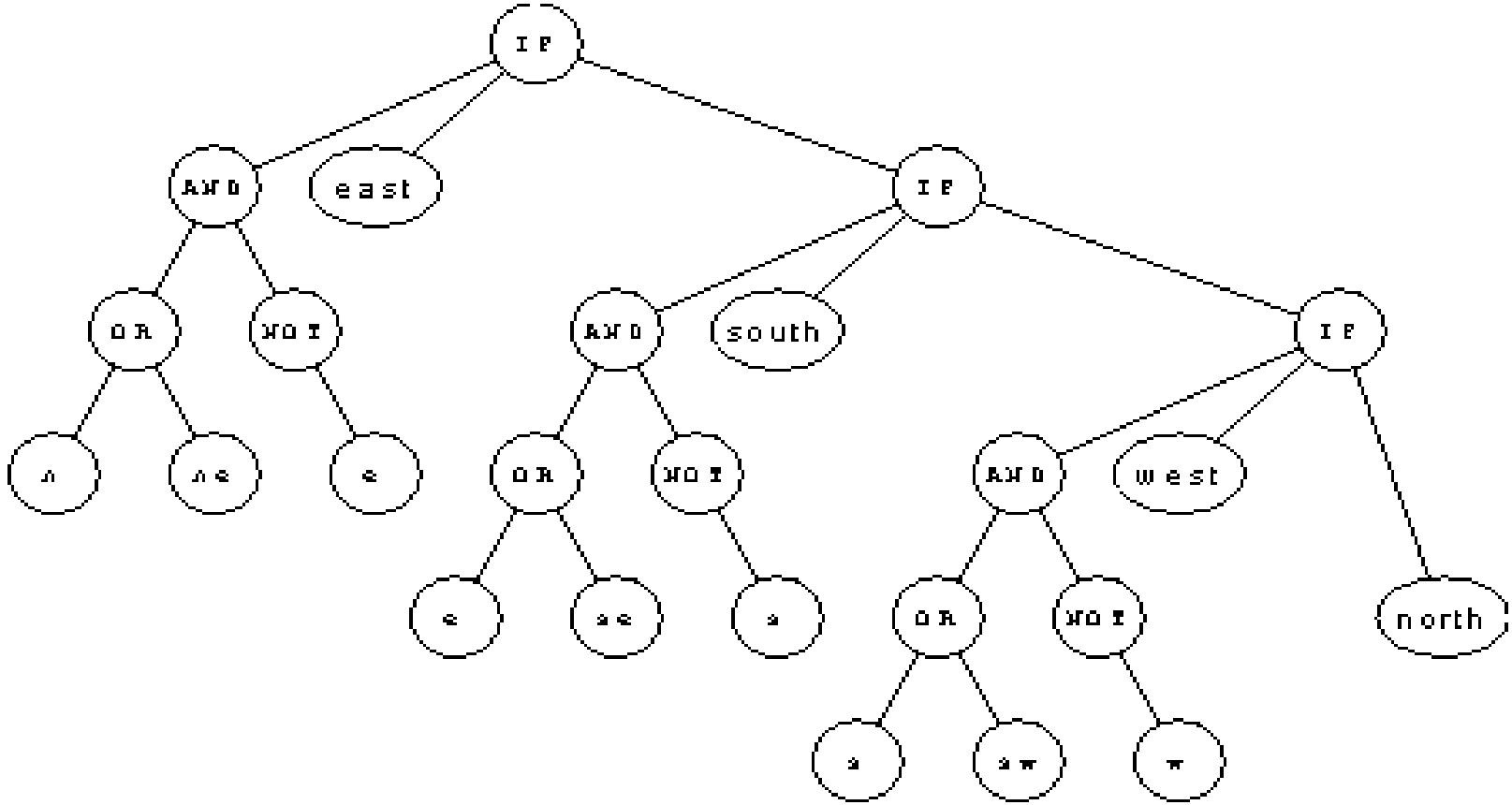
  - *IF(x,y,z)=y if x=1; else z*

  4가지 동작

  (north, east, south, west)



벽을 따라가는 로봇을 위한 GP



(IF (AND (OR (n) (ne)) (NOT (e)))
(east)
(IF (AND (OR (e) (se)) (NOT (s)))
(south)
(IF (AND (OR (s) (sw)) (NOT (w)))
(west)
(north))))

청소 로봇 사례의 평가 조건 읽기

입력: 주변 8방향 센서와 기본 명령

개체군 5,000개의 프로그램을 시험

10개 시작 위치, 각 위치에서 최대 32점

최고 적합도 320점

한 환경에서 320점이어도 다른 지도 검증 필요

2 GP 과정

- 임의의 프로그램으로 구성된 개체군으로 시작 (a population of random programs, (*Generation 0*))
  - 함수, 상수, 센서 입력을 사용
- Population Size : 5000 (programs)
- 적합도 (평가) 함수 :
  - 각 프로그램을 60 번 실행
  - 이 60단계 동안에 벽에 인접하게 된 cell 수를 셈(max : 32).
  - 임의로 생성된 10개의 시작 위치에서 이 과정을 10번 반복
  - 10번 실행으로 얻은 벽에 근접한 셀(next-to-the-wall cell) 수의 합이 프로그램의 적합도(fitness)
  - 가능한 가장 높은 적합도 값은 320

벽을 따라가는 청소로봇을 위한 GP

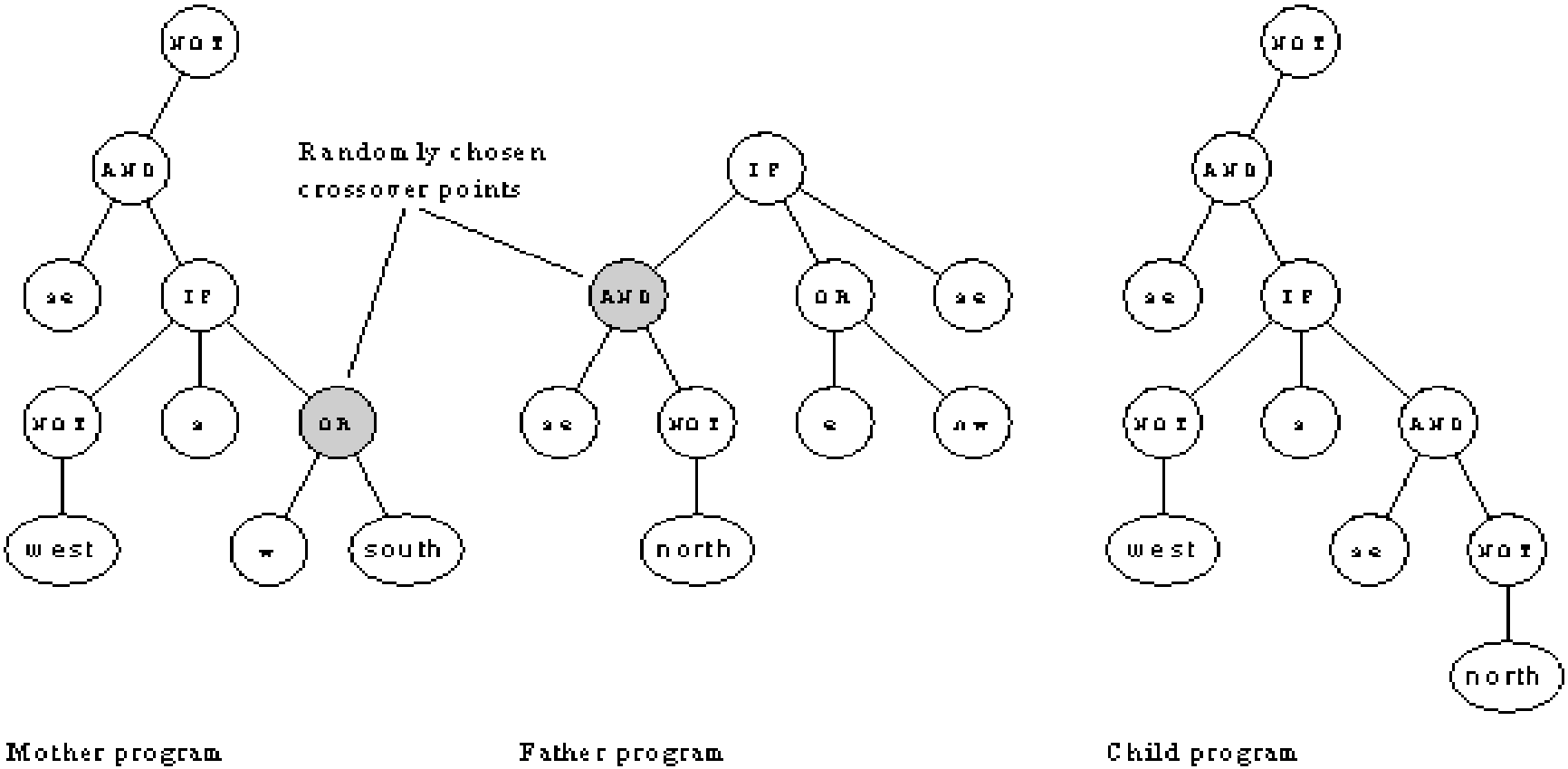
- i번째 세대로부터 (i+1)번째 세대를 만드는 방법

1. i번째 세대에서 500개의 프로그램(10%)이 i+1 번째 세대로 직접 복제됨
   토너먼트 선택(*Tournament selection process*):
   7개의 프로그램이 5000개의 개체군중에서
   임의로 선택되고 이 중에서 가장 좋은 것을 선택

2. 4500개의 새로운 자식 프로그램(90%)이 i+1 번째 세대에 추가됨
   각각의 자식 프로그램은 부모의 교차 연산(*crossover operation*)으로 생성
   어머니 프로그램과 아버지 프로그램은 토너먼트 선택(*tournament selection*)으로 추출

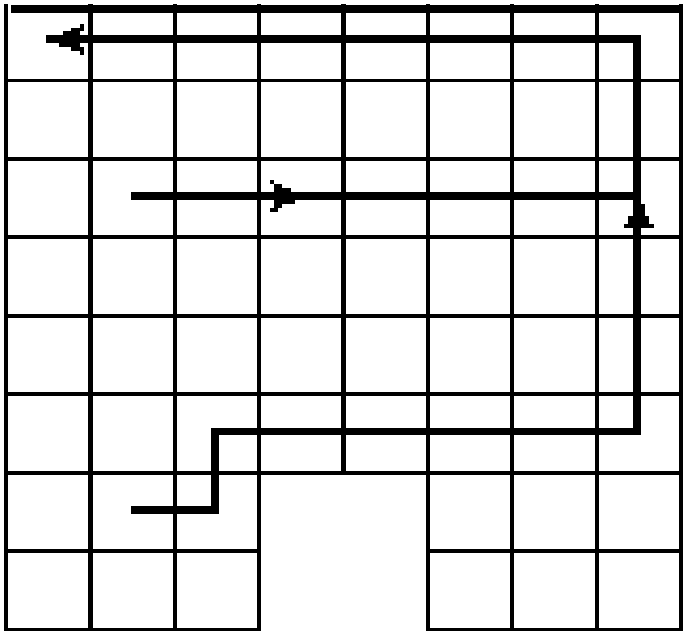
벽을 따라가는 청소로봇을 위한 GP

교차 연산(crossover operator) :



3 벽을 따라가는 청소로봇을 진화시키기

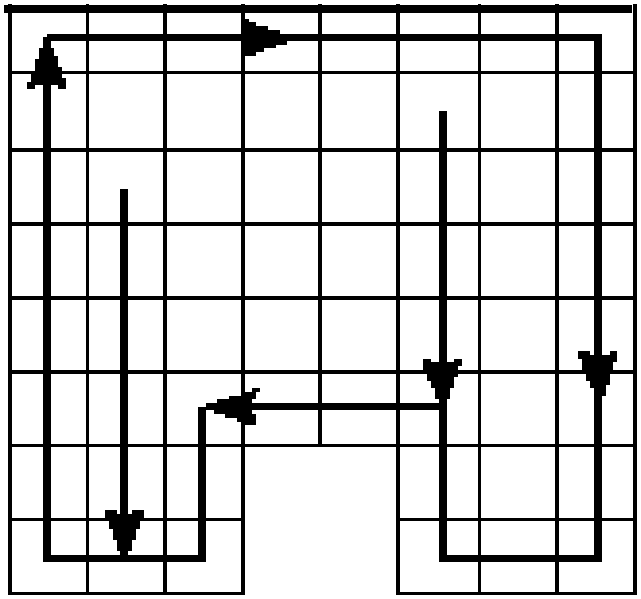
Generation 0 : (fitness=92)



```
(AND (NOT (NOT (IF (IF (NOT (nw))
                       (IF (e)(north) (east))
                       (IF (west)(0) (south))
                   (OR (IF (nw)(ne)(w))
                       (NOT (sw)))
                   (NOT (NOT (north))))))
     (IF (OR (NOT (AND (IF (sw)(north)(ne))
                       (AND (south)(1))))
             (OR (OR (NOT (s))
                     (OR (e)(e)))
                 (AND (IF (west)(ne)(se))
                      (IF (1) (e)(e)))))
         (OR (NOT (AND (NOT (ne))(IF (east)(s)(n))))
             (OR (NOT (IF (nw)(east)(s)))
                 (AND (IF (w)(sw)(1))
                      (OR (sw)(nw))))))
         (OR (NOT (IF (OR (n)(w))
                      (OR (0)(se))
                      (OR (1)(east))))
             (OR (AND (OR (1)(ne))
                      (AND (NW)(east)))
                 (IF (NOT (west))
                     (AND (west)(east))
                     (IF (1)(north)(w)))))))
```

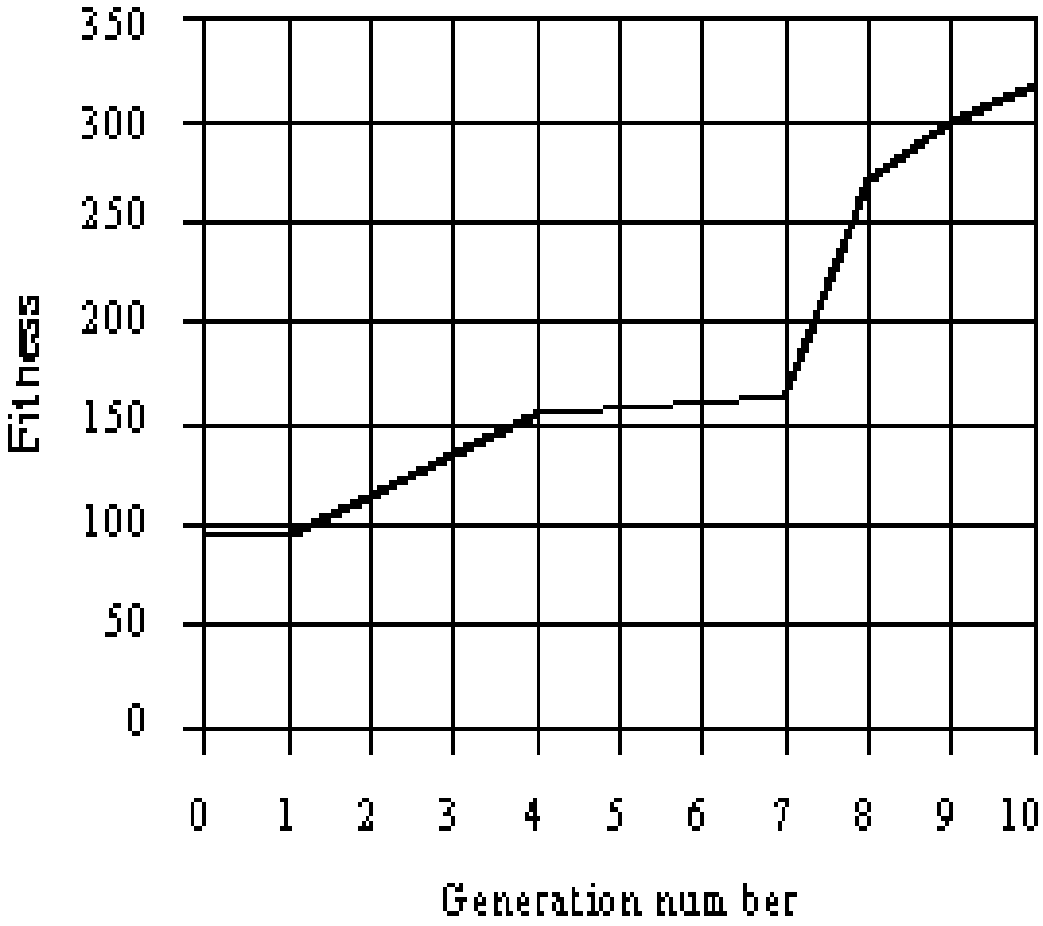
벽을 따라가는 청소로봇을 진화시키기

Generation 10 : (fitness=320)



```
(IF (IF (IF (se)(0)(ne))
        (OR (se)(east))
        (IF (OR (AND (e)(0))
                (sw))
            (OR (sw)(0))
            (AND (NOT (NOT (AND (s)(se))))
                 (se))))
    (IF (w)
        (OR (north)
            (NOT (NOT (s))))
        (west))
    (NOT (NOT (NOT (AND (IF (NOT (south))
                            (se)
                            (w))
                        (NOT (n)))))))
```

벽을 따라가는 로봇을 진화시키기



훈련 적합도와 새 지도 성능

세대 0: 92점, 세대 10: 320점 (같은 평가 조건)

훈련 시작 위치를 기억했을 가능성도 있다

새 지도·시작 위치·센서 잡음에서 재평가

트리 크기와 실행 시간을 함께 기록

적합도 상승만으로 일반화를 단정하지 않는다

오늘날 GP 활용 예: 수식 찾기

데이터 (x,y)에서 작은 수식 트리를 탐색

예: 게임 물리 로그의 이동량을 식으로 근사

오차와 식의 복잡도를 함께 고려

NEAT(NeuroEvolution of Augmenting Topologies: 진화 알고리즘 계열의 **신경진화(neuroevolution)** 방법) 등 신경망 진화는 별도 접근

해석 가능한 식도 새 데이터에서 검증

실습 | 학습 점수만 높으면 성공인가?

맵 A의 10개 출발점에서는 320/320

새 맵 B에서는 12/320

이 로봇의 GP 실험을 어떻게 고칠까?

평가 기준과 변이 연산을 함께 생각

Summary

- 유전자 알고리즘은 후보 해의 표현, 적합도 평가, 선택, 교차·돌연변이, 세대 교체를 반복한다.
- 문제에 따라 비트열·순열·트리를 쓰며, 연산자는 표현의 제약을 지켜야 한다.
- 적합도 설계와 평가 예산, 여러 시드 및 새로운 환경에서의 검증이 중요하다.

Q & A

